



UNIVERSIDADE FEDERAL DE RORAIMA CAMPUS PARICARANA

CENTRO DE CIÊNCIAS TECNOLÓGICAS

CURSO: Ciência da Computação

DISCIPLINA: Arquitetura e Organização de Computadores 2026.2

Docente: Prof. Dr. Herbert Oliveira Rocha

Discente: Mateus Henrique de Souza Pinheiro, Darckison Almeida Trajano

Nº Matrícula: 2023010576 - 2021022845

Repositório GitHub:

https://github.com/MateusHenrique2025/AOC_MateusHenrique_DarckisonAlmeida_UFRR_MIPS_WA_SM_2026

Relatório

BOA VISTA - RR 2026





Relatório

Este relatório descreve o desenvolvimento e a simulação da aritmética da arquitetura MIPS direcionados à plataforma WebAssembly. O trabalho foca na manipulação em nível de bit utilizando a linguagem C, na aplicação de Verificação Formal de Software (via Map2Check) para garantir a integridade do código contra falhas críticas, e na posterior compilação para WebAssembly (WASM). Por fim, apresenta-se uma interface gráfica interativa que ilustra o fluxo aritmético passo a passo, consolidando de forma prática os conceitos teóricos abordados ao longo da disciplina.

1 RELATÓRIO

1.1 Resumo

Este relatório descreve a implementação e simulação das operações aritméticas da arquitetura MIPS voltadas para WebAssembly. O trabalho contempla a manipulação em nível de bit utilizando a linguagem C, a garantia de integridade contra falhas críticas via Verificação Formal de Software com o Map2Check, e a compilação do código para uma máquina virtual baseada em pilha (WASM). Por fim, apresenta-se uma interface gráfica interativa para a animação do fluxo aritmético passo a passo, consolidando os conceitos teóricos e práticos abordados ao longo da disciplina.

2 RESPOSTAS AOS QUESTIONAMENTOS TEÓRICOS

2.1 Pergunta 1: Comparação do fluxo de instruções (.wat) e manipulação de variáveis (WASM vs MIPS)

Ao analisar o arquivo `.wat` gerado para a função `soma`, percebe-se uma diferença fundamental na manipulação de dados em relação ao MIPS. Enquanto o assembly MIPS exige a especificação explícita dos registradores de origem e destino (como em `add $t0, $t1, $t2`), o WebAssembly opera com base em uma máquina de pilha. No `.wat`, as variáveis locais não dependem de registradores fixos como `$t0` ou `$t1`; em vez disso, são carregadas para o topo da pilha de execução por meio de instruções `local.get`. Na sequência, a instrução `i32.add` é executada sem passar parâmetros explícitos, pois ela retira (*pop*) automaticamente os dois últimos valores empilhados, realiza a soma e insere (*push*) o resultado de volta no topo da pilha.

2.2 Pergunta 2: Simulação de comportamento da arquitetura e custo computacional (Hardware vs Software)

A função `soma_segura` simula a instrução `add` do MIPS implementando a detecção de *overflow* em complemento de dois diretamente via software, sinalizando o estouro através de uma *flag* booleana em vez de uma interrupção por hardware. Esse tratamento em C/WASM é consideravelmente mais custoso: enquanto o MIPS identifica o *overflow* no próprio circuito da ALU dentro do ciclo da instrução, a máquina virtual do WASM precisa buscar e executar um conjunto adicional de instruções lógicas e desvios condicionais (*branches*) apenas para validar se o limite de 32 bits foi ultrapassado.

2.3 Pergunta 3: Análise do contraexemplo (Map2Check) e a falha em Complemento de Dois

Explique os valores do contraexemplo gerado pelo Map2Check. Como a representação em complemento de dois explica a falha identificada pela ferramenta?

Contraexemplo do Map2Check

O Map2Check encontra uma falha na função soma básica ao fornecer valores nos limites dos inteiros de 32 bits com sinal (`int32_t`), cujo intervalo válido vai de $-2.147.483.648$ (2^{31}) até $2.147.483.647$ ($2^{31}-1$). Um contraexemplo típico gerado pela ferramenta é a soma do valor máximo com um número positivo (por exemplo, $a = 2147483647$ e $b = 1$). Como a função simples apenas executa `return a + b`; sem qualquer verificação, a ferramenta aponta que o resultado obtido não corresponde matematicamente à soma real esperada.

Complemento de dois

Na representação em complemento de dois de 32 bits, o bit mais significativo (MSB, bit 31) define o sinal do número: **0** para positivo e **1** para negativo.

- O maior número positivo ($2.147.483.647$) em binário é **01111111 11111111 11111111 11111111**.
- Ao somar **1** (**00000000 00000000 00000000 00000001**), o "vai-um" (*carry*) propaga-se ao longo dos bits e altera o MSB de **0** para **1**, resultando em **10000000 00000000 00000000 00000000**.
- Na arquitetura de hardware/VM, esse padrão de bits é interpretado como o menor número negativo possível ($-2.147.483.648$).

A falha ocorre porque a operação ultrapassa a capacidade máxima de representação de inteiros positivos de 32 bits, fazendo com que o valor "dê a volta" (*wrap-around*) para o campo dos números negativos, o que é identificado pelo Map2Check como uma violação de *overflow*.